Decoders, Encoders, and Demultiplexers

Emmanuel Cano

Goal of the Exercise

The goal of this exercise was to design and understand how decoders, encoders, and demultiplexers work in digital systems. These components are commonly used for data routing, control signals, and memory addressing. We also learned how to design and test their functionality using VHDL and ModelSim simulation software. Additionally, we designed a combinational circuit (encoder and decoder system) using VHDL and ModelSim.

Tools and Environment

- HDL: VHDL
- FPGA Toolchain: Intel Quartus Prime Lite
- Simulator: ModelSim
- Target Platform: Simulation-only (no hardware deployment)

VHDL Implementation

```
library ieee;
use ieee.std_logic_1164.all;

entity Decoder3to8 is
    Port (
        A2 : in std_logic;
        A1 : in std_logic;
        A0 : in std_logic;
        Y0 : out std_logic;
        Y1 : out std_logic;
        Y2 : out std_logic;
        Y3 : out std_logic;
        Y4 : out std_logic;
        Y5 : out std_logic;
        Y6 : out std_logic;
        Y7 : out std_logic
    );
end entity;

architecture Decoder3to8Arch of Decoder3to8 is
begin
    process(A2, A1, A0)
                    variable A_vec : std_logic_vector(2 downto 0);
    begin
                A_vec := A2 & A1 & A0;

        Y0 <= '0';
```

```
        Y1 <= '0';
        Y2 <= '0';
        Y3 <= '0';
        Y4 <= '0';
        Y5 <= '0';
        Y6 <= '0';
        Y7 <= '0';

        case A_vec is
            when "000" => Y0 <= '1';
            when "001" => Y1 <= '1';
            when "010" => Y2 <= '1';
            when "011" => Y3 <= '1';
            when "100" => Y4 <= '1';
            when "101" => Y5 <= '1';
            when "110" => Y6 <= '1';
            when "111" => Y7 <= '1';
            when others => null;
        end case;
    end process;
end architecture;
```

Figure 1. *3:8 Decoder*

Figure 1's 3:8 decoder design works by activating one of eight outputs depending on the 3-bit input. When the input is binary 101, for example, only output Y5 is set to '1'. All others are set to '0'. For convenience, the binary input is matched to its decimal representation output. All components in this exercise were implemented using the behavioral modeling style for clarity and simplicity. Furthermore, all designs in this exercise are purely combinational and do not use clocked logic.

```
library ieee;
use ieee.std_logic_1164.all;

entity Priority_Encoder8to3 is
    Port (
        D0, D1, D2, D3, D4, D5, D6, D7 : in std_logic;
        Y0, Y1, Y2 : out std_logic;
        valid : out std_logic
    );
end entity;

architecture Priority_Encoder8to3Arch of Priority_Encoder8to3 is
begin
    process(D0, D1, D2, D3, D4, D5, D6, D7)
    begin
        Y0 <= '0';
        Y1 <= '0';
        Y2 <= '0';
        valid <= '0';

        if D7 = '1' then
                            Y2 <= '1'; Y1 <= '1'; Y0 <= '1';
            valid <= '1';
        elsif D6 = '1' then
            Y2 <= '1'; Y1 <= '1'; Y0 <= '0';
```

```
            valid <= '1';
        elsif D5 = '1' then
            Y2 <= '1'; Y1 <= '0'; Y0 <= '1';
            valid <= '1';
        elsif D4 = '1' then
            Y2 <= '1'; Y1 <= '0'; Y0 <= '0';
            valid <= '1';
        elsif D3 = '1' then
            Y2 <= '0'; Y1 <= '1'; Y0 <= '1';
            valid <= '1';
        elsif D2 = '1' then
            Y2 <= '0'; Y1 <= '1'; Y0 <= '0';
            valid <= '1';
        elsif D1 = '1' then
            Y2 <= '0'; Y1 <= '0'; Y0 <= '1';
            valid <= '1';
        elsif D0 = '1' then
            Y2 <= '0'; Y1 <= '0'; Y0 <= '0';
            valid <= '1';
        end if;
    end process;
end architecture;
```

Figure 2. *8:3 Priority Encoder*

Figure 2's 8:3 priority encoder circuit encodes which input line is active, giving priority to the highest bit. If multiple inputs are high, the output corresponds to the most significant active bit. To maintain priority, 'if' and 'elsif' are used instead of 'when', and start from D7 to D0. For convenience, the input decimal representation is matched to its decimal representation output. For example, if input line D5 is active the output will be '101'. Furthermore, if multiple input lines were active, say D3, D6, and D7, the output would be '111' for D7 since it has highest-order priority.

```
library ieee;
use ieee.std_logic_1164.all;

entity Demux1to2 is
    Port (
        D : in std_logic;
        S : in std_logic;
        Y0 : out std_logic;
        Y1 : out std_logic
    );
end entity;

architecture Demux1to2Arch of Demux1to2 is
begin
    process(D, S)
    begin
        if S = '0' then
            Y0 <= D;
            Y1 <= '0';
        else
            Y0 <= '0';
            Y1 <= D;
        end if;
```

```
    end process;
end architecture;
```

Figure 3. *1:2 Demultiplexer*

Figure 3's 1:2 demux design sends the input data to one of two outputs depending on the select line. In this case, if S = '0', data goes to output Y0 or if S = '1', it goes to Y1`. The output that isn't selected is set to '0'. Similar to the 2:1 mux, the output is dependent on the select line S.

```
library ieee;
use ieee.std_logic_1164.all;

entity Encoder_Decoder_System is
    Port (
        D_in0 : in std_logic;
        D_in1 : in std_logic;
        D_in2 : in std_logic;
        D_in3 : in std_logic;
        D_in4 : in std_logic;
        D_in5 : in std_logic;
        D_in6 : in std_logic;
        D_in7 : in std_logic;

        D_out0 : out std_logic;
        D_out1 : out std_logic;
        D_out2 : out std_logic;
        D_out3 : out std_logic;
        D_out4 : out std_logic;
        D_out5 : out std_logic;
        D_out6 : out std_logic;
        D_out7 : out std_logic
    );
end entity;

architecture
Encoder_Decoder_SystemArch of
Encoder_Decoder_System is
    signal encoded0 : std_logic;
    signal encoded1 : std_logic;
    signal encoded2 : std_logic;
    signal valid : std_logic;

    component Priority_Encoder8to3
        Port (
            D0 : in std_logic;
            D1 : in std_logic;
            D2 : in std_logic;
            D3 : in std_logic;
            D4 : in std_logic;
            D5 : in std_logic;
            D6 : in std_logic;
            D7 : in std_logic;

            Y0 : out std_logic;
            Y1 : out std_logic;
            Y2 : out std_logic;
```

```
            valid : out std_logic
        );
    end component;

    component Decoder3to8
        Port (
            A0 : in std_logic;
            A1 : in std_logic;
            A2 : in std_logic;

            Y0 : out std_logic;
            Y1 : out std_logic;
            Y2 : out std_logic;
            Y3 : out std_logic;
            Y4 : out std_logic;
            Y5 : out std_logic;
            Y6 : out std_logic;
            Y7 : out std_logic
        );
    end component;
begin
    ENC: Priority_Encoder8to3
        port map(
            D0 => D_in0,
            D1 => D_in1,
            D2 => D_in2,
            D3 => D_in3,
            D4 => D_in4,
            D5 => D_in5,
            D6 => D_in6,
            D7 => D_in7,

            Y0 => encoded0,
            Y1 => encoded1,
            Y2 => encoded2,

            valid => valid
        );

    DEC: Decoder3to8
        port map(
            A0 => encoded0,
            A1 => encoded1,
            A2 => encoded2,

            Y0 => D_out0,
            Y1 => D_out1,
            Y2 => D_out2,
            Y3 => D_out3,
            Y4 => D_out4,
            Y5 => D_out5,
            Y6 => D_out6,
            Y7 => D_out7
        );
end architecture;
```

Figure 4. *Interconnected Device (Encoder + Decoder)*

Here in Figure 4's design, the encoder takes in an 8-bit input that is encoded based on the input line activated. The encoder's 3-bit output then connects to the decoder's 3-bit input. As a result, its 8-bit output is supposed to represent the same 8-bit input that the encoder received. Since

the encoder used in this design is a priority encoder, when multiple input lines are active, it only sends the highest-order priority to the decoder which is what it outputs. This design demonstrates how data can be encoded, transmitted, and then decoded back.

Block Diagrams

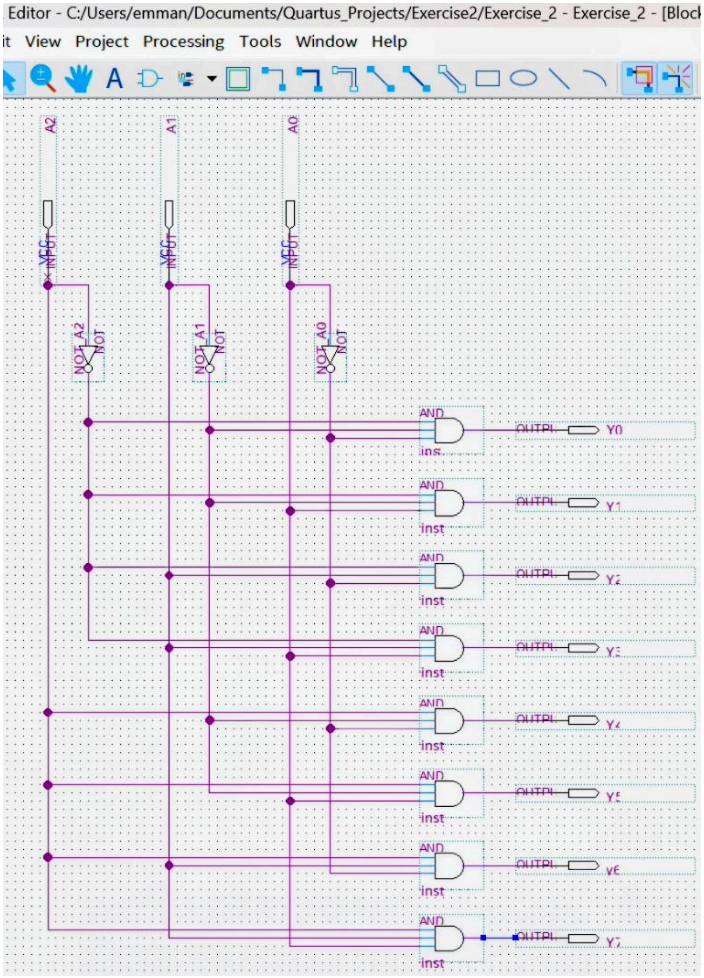

Figure 5. *Block diagram of the 3:8 Decoder*

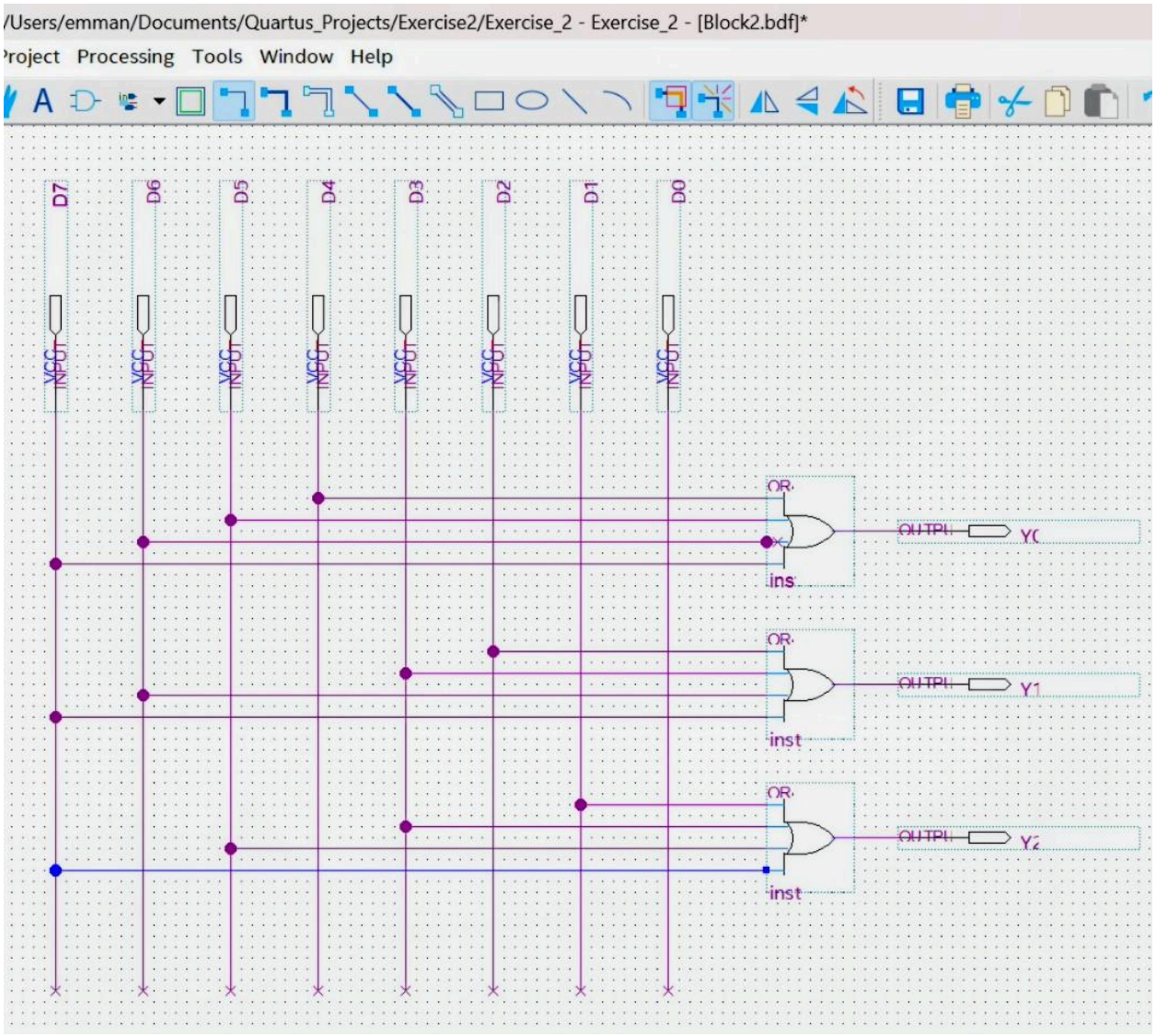

Figure 6. *Block diagram of the 8:3 Encoder*

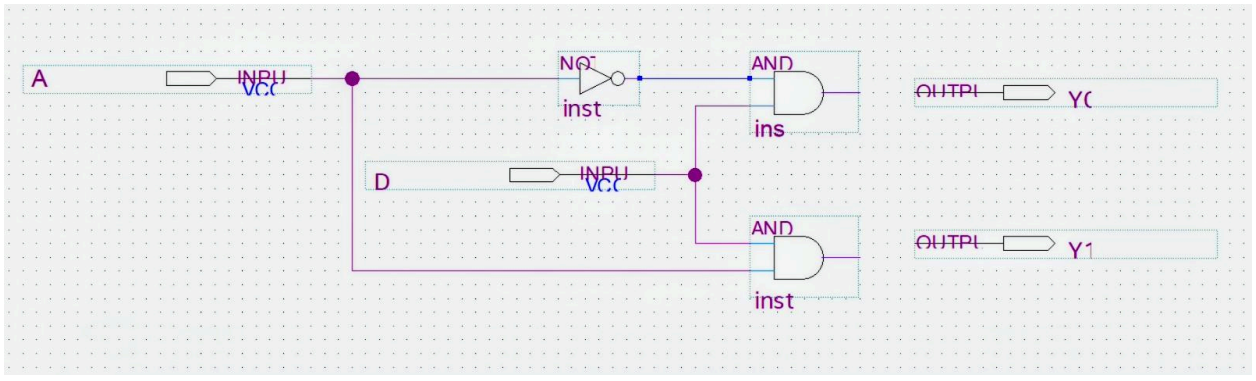

Figure 7. *Block diagram of the 1:2 Demultiplexer*

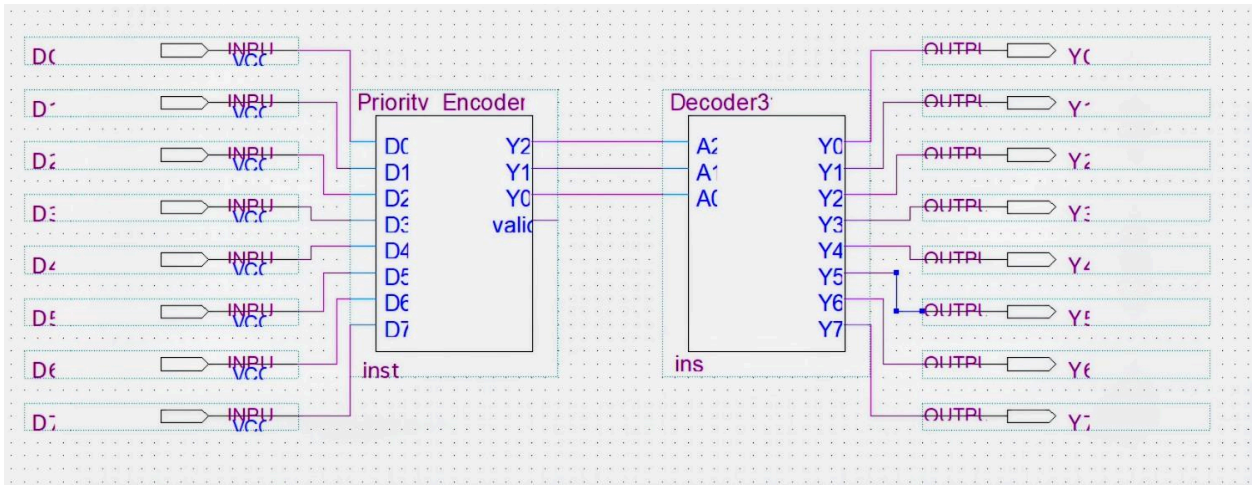

Figure 8. *Block diagram of the Interconnected Device (Encoder + Decoder)*

ModelSim Simulation Results

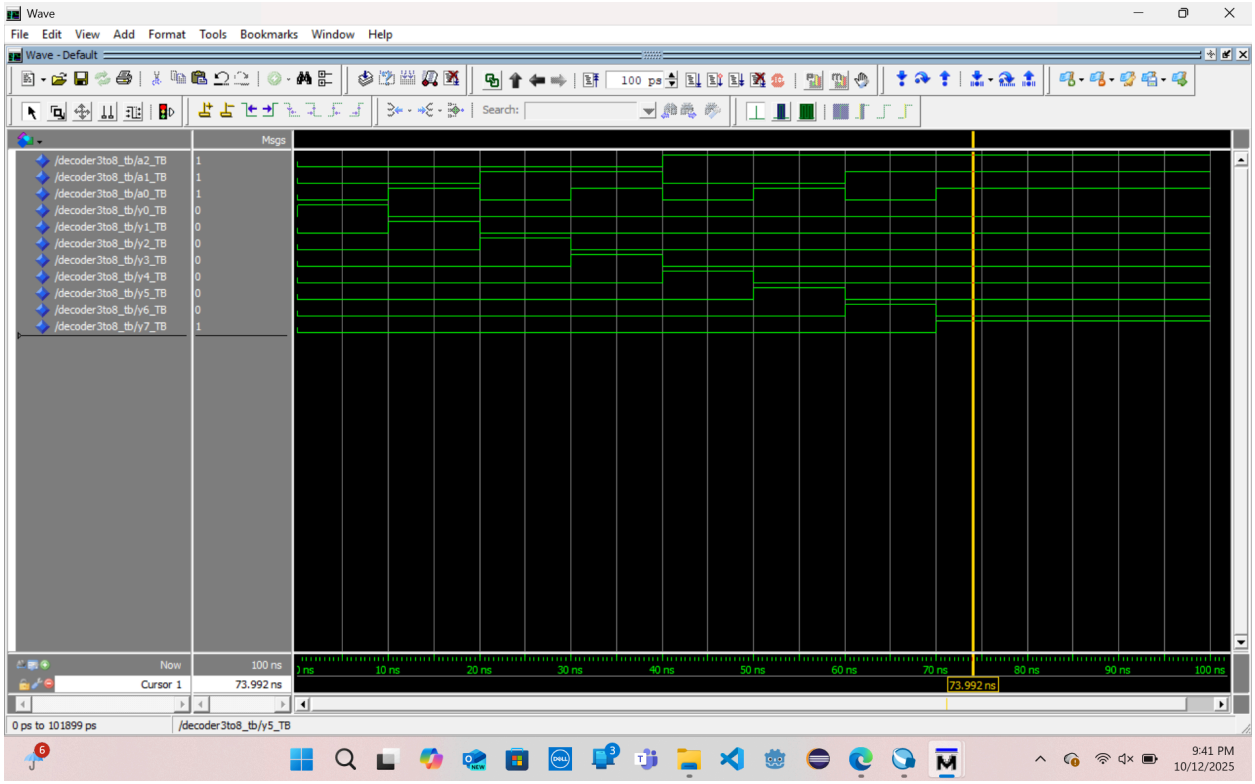

Figure 9. *Simulation results of the 3:8 Decoder (showing outputs activating one at a time).*

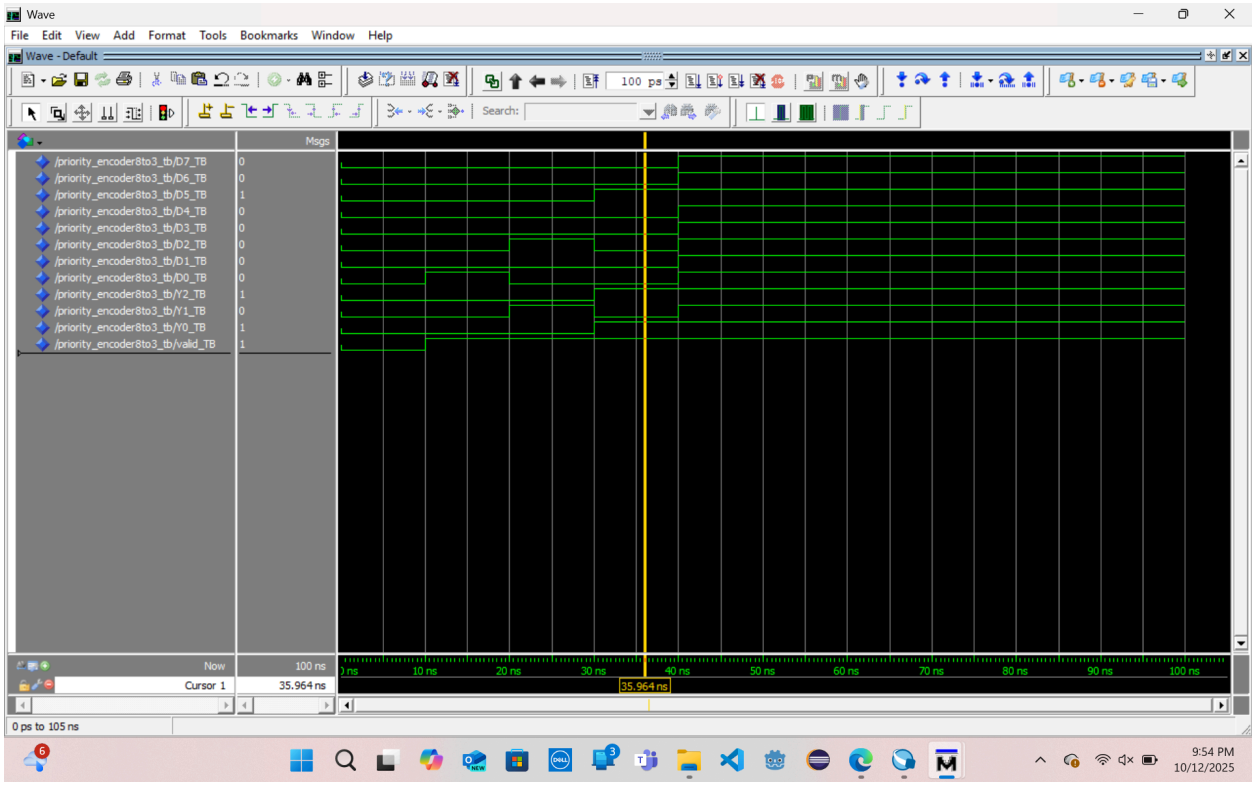

Figure 10. *Simulation results of the 8:3 Encoder (showing priority resolution).*

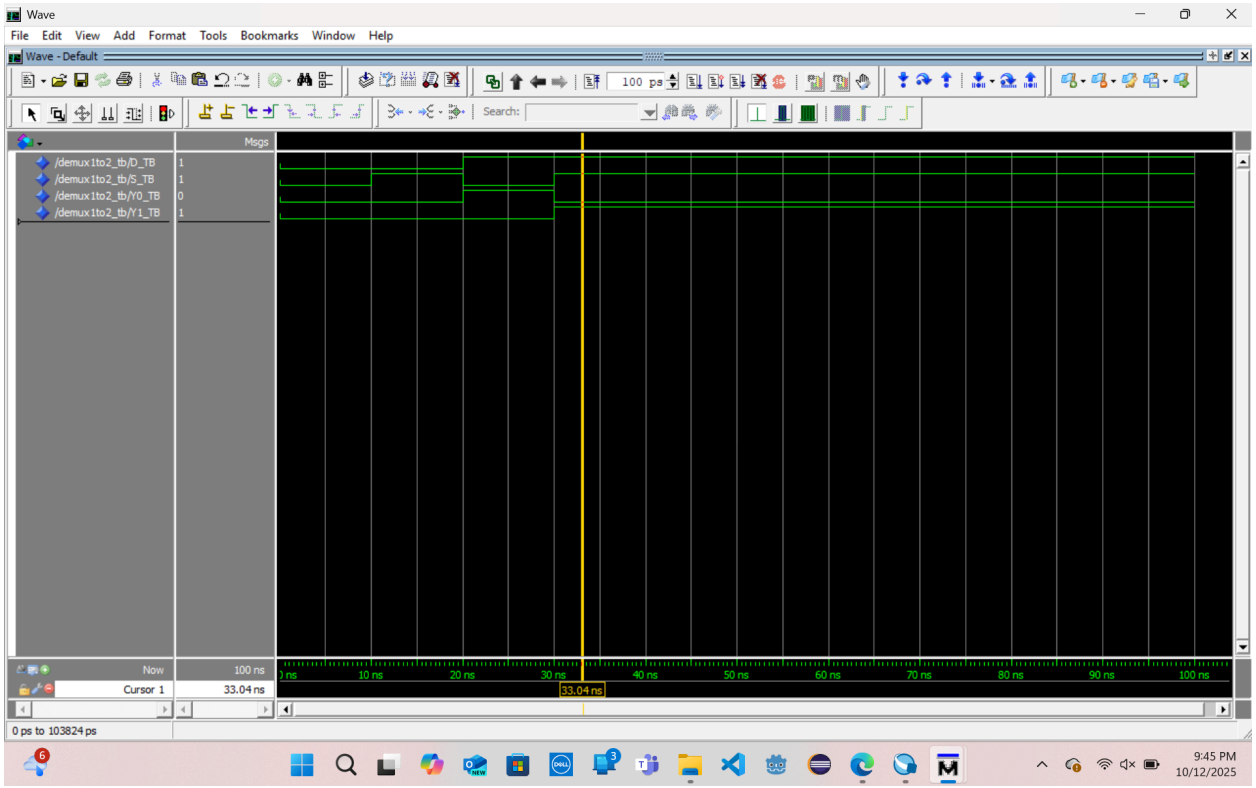

Figure 11. *Simulation results of the 1:2 Demultiplexer (data routed according to select line).*

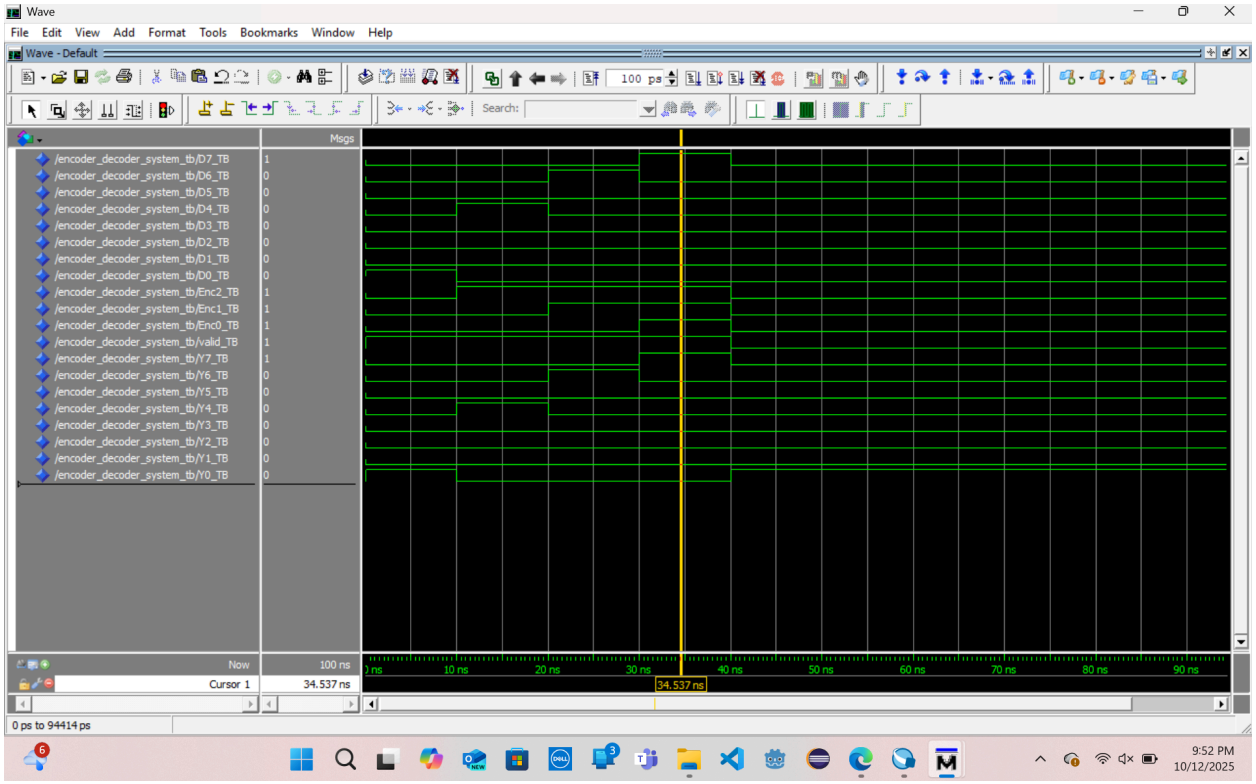

Figure 12. *Simulation results of the Interconnected Device (Encoder + Decoder, showing correct round-trip output).*

Conclusion

In this exercise, we successfully designed and simulated a decoder, a priority encoder, and a demultiplexer using VHDL, as well as the combinational circuit. The decoder showed how binary inputs can select one of many outputs, while the priority encoder showed how multiple requests can be simplified into a single binary representation. The demultiplexer illustrated how a single input can be distributed based on control signals. Finally, connecting the encoder and decoder together proved that the encoded output could be successfully decoded back to the correct line. Through simulation in ModelSim, the functionality of all designs was confirmed and the results matched the truth tables. This exercise helped us understand what goes on in digital systems whose function is data routing, control signals, and memory addressing.